Reviewer Pack — Submodular Rank Gap in Monotone DNF for k-CLIQUE

Entry 6ae57c36380b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 18:22:44 UTC

Submodular Rank Gap in Monotone DNF for k-CLIQUE

Entry ID: `6ae57c36380b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-12 18:22:44 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory (Polymatroid Expansion) **Field B** (complexity object): Monotone Circuit Size for k-CLIQUE

Statement:

> For any k-CLIQUE instance with n ≥ 40, the submodular rank of its characteristic set system is Ω(n), while any monotone DNF formula of size O(poly(n)) has submodular rank O(log n).

Rationale (proposer's reasoning):

> Polymatroid rank captures the combinatorial structure of k-CLIQUE's hypergraph, and its submodular gap may reflect the inherent complexity of monotone computation. The conjecture links matroid-theoretic invariants to circuit size via the polymatroid expansion of the problem's incidence matrix.

Taxonomy category: `MONOTONE_CLIQUE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b6f82c9ce9d11c6e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_clique(n, k):
        vertices = list(range(n))
        edges = set()
        for combo in combinations(vertices, 2):
            if len(set(combo)) == 2 and random.random() < (k / n):
                edges.add(tuple(sorted(combo)))
        return vertices, edges

    def incidence_matrix(vertices, edges):
        n = len(vertices)
        m = len(edges)
        A = [[0] * m for _ in range(n)]
        edge_dict = {edge: i for i, edge in enumerate(edges)}
        for v in vertices:
            for u in vertices:
                if (v, u) in edges or (u, v) in edges:
                    A[v][edge_dict[(min(v, u), max(v, u))]] = 1
        return A

    def submodular_rank(A):
        n = len(A)
        m = len(A[0])
        rank = 0
        for i in range(m):
            max_col = -1
            max_val = -float('inf')
            for j in range(n):
                if A[j][i] > max_val:
                    max_val = A[j][i]
                    max_col = j
            if max_col == -1:
                break
            rank += 1
            for j in range(n):
```

```
            if j != max_col:
                A[j][i] -= A[j][max_col] * A[max_col][i] / A[max_col][max_col]
    return rank

def dpll_solver(A, assignment, clause_count):
    n = len(A)
    m = len(A[0])
    if all(assignment[i] is not None for i in range(n)):
        return True
    var = next(i for i in range(n) if assignment[i] is None)
    for val in [True, False]:
        new_assignment = assignment[:]
        new_assignment[var] = val
        if dpll_solver(A, new_assignment, clause_count):
            return True
    return False

def min_dnf_size(vertices, edges):
    n = len(vertices)
    m = len(edges)
    max_clauses = 2 ** n - 1
    for clause_count in range(1, max_clauses + 1):
        if dpll_solver(incidence_matrix(vertices, edges), [None] * n, clause_count):
            return clause_count
    return float('inf')

def is_monotone_dnf(A):
    n = len(A)
    m = len(A[0])
    for i in range(m):
        max_col = -1
        max_val = -float('inf')
        for j in range(n):
            if A[j][i] > max_val:
                max_val = A[j][i]
                max_col = j
        if max_col == -1:
            break
        for j in range(n):
            if j != max_col and A[j][max_col] < A[j][i]:
                return False
    return True

n = 40
k = random.randint(2, 5)
vertices, edges = generate_k_clique(n, k)
A = incidence_matrix(vertices, edges)

rank = submodular_rank(A)
dnf_size = min_dnf_size(vertices, edges)
is_monotone = is_monotone_dnf(A)

return {
    "metric_name": "submodular_rank",
    "metric_value": rank,
    "instances_tested": 1,
```

```
        "conjecture_holds": rank >= 0.1 * n and dnf_size <= n**2 and (not is_monotone or dnf_size
        "counterexample": "" if rank >= 0.1 * n and dnf_size <= n**2 and (not is_monotone or dnf_s
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(2, 6

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fracti
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print(f"RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e89e0fcc.py", line 120, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e89e0fcc.py", line 102, in run_trial
    rank = submodular_rank(A)
           ^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e89e0fcc.py", line 55, in submodular_ra
    A[j][i] -= A[j][max_col] * A[max_col][i] / A[max_col][max_col]
               ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^~~~~~~~~~~~~~~~~~~~~
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with ZeroDivisionError, preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Debug the ZeroDivisionError in submodular_rank computation by adding numerical stability (e.g., epsilon regularization) and retesting

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	47857
2	propose	ollama_remote	qwen3:8b	0	0	22755
3	preregistration	ollama_remote	qwen3:8b	0	0	23889
4	novelty	ollama_remote	qwen3:8b	0	0	22226
5	novelty	ollama_remote	qwen3:8b	0	0	16520
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24799
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13135
8	judge	ollama_remote	qwen3:8b	0	0	17647

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 188829 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/6ae57c36380b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6ae57c36380b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/6ae57c36380b.tar.gz` (if generated)